FUNDSCOPE

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.0

Campo	Descrição
Projeto	FundScope - Pipeline e Data Warehouse de Fundos de Investimento da CVM
Gerente de Projeto	Arthur Faust Locatelli
Gerente/Responsável de GCS-DevOps	Arthur Faust Locatelli
Fábrica de Software / Squad	Projeto Individual
Repositório principal	https://github.com/arthurfaust/fundscope

Histórico de Revisões

Data	Versão	Descrição	Autor
10/08/2026	1.0	Elaboração do documento com enfoque em práticas DevOps (CI/CD).	Arthur Faust

Sumário

1. Introdução **3**
1.1 Finalidade 3
1.2 Escopo 3
1.3 Definições, Acrônimos e Abreviações 4
1.4 Referências 5
1.5 Visão Geral 5
2. Cultura DevOps e Papéis na Gerência de Configuração **5**
3. Ferramentas, Ambientes e Infraestrutura **6**
3.1 Ferramentas do Pipeline 6
3.2 Ambientes e Infraestrutura 6
3.3 Estratégia de Branching e Versionamento 7
4. Identificação da Configuração **8**
4.1 Itens de Configuração 8
4.2 Convenção de Nomenclatura 8
4.3 Estrutura do Repositório 9
5. Integração Contínua (CI) **9**
5.1 Etapas do Pipeline de CI 10
5.2 Política de Pull Request e Revisão de Código 10
5.3 Critérios de Qualidade (Quality Gate) 10
6. Entrega e Implantação Contínua (CD) **10**
6.1 Etapas do Pipeline de CD 11
6.2 Estratégias de Implantação 11
6.3 Rollback e Recuperação 11
7. Infraestrutura como Código (IaC) **12**
7.1 Versionamento de Infraestrutura 12
7.2 Ambientes Efêmeros 12
8. Controle de Configuração e Mudança **12**
8.1 Processo de Solicitação de Mudança 12
8.2 Comitê de Controle de Mudança (CCM) 13
9. Baselines e Releases **13**
9.1 Geração Automática de Baseline 13
9.2 Notas de Versão (Release Notes) 13
10. Estimativa do Status de Configuração **13**
10.1 Monitoramento e Observabilidade 13
10.2 Métricas DORA 14
10.3 Relatórios e Auditorias 14
11. Marcos **14**
12. Treinamento e Recursos **15**
13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros **15**
14. Quadro-Resumo do Pipeline CI/CD **16**

1. Introdução

O FundScope é um projeto acadêmico de engenharia de dados destinado à coleta, processamento, armazenamento e disponibilização de informações públicas sobre fundos de investimento. O sistema utilizará dados cadastrais e informes diários publicados pela Comissão de Valores Mobiliários (CVM), considerando inicialmente os últimos 12 meses.

Os dados serão obtidos por meio de um pipeline automatizado, passando pelas etapas de extração, validação, transformação e carga. Os arquivos originais serão preservados na camada Bronze, os dados tratados e padronizados serão disponibilizados na camada Silver e as informações preparadas para análise serão organizadas na camada Gold.

O projeto utilizará Python e Pandas para processamento dos dados, PostgreSQL para persistência, Apache Airflow para orquestração das cargas, Docker para criação do ambiente reproduzível e Streamlit para apresentação das informações. O código-fonte e os demais itens de configuração serão versionados com Git e GitHub.

1.1 Finalidade

Este Plano de Gerenciamento de Configuração tem como finalidade estabelecer os procedimentos utilizados para identificar, versionar, testar, controlar e disponibilizar os componentes do FundScope.

O plano busca garantir que alterações realizadas no código-fonte, nas DAGs do Apache Airflow, no modelo de dados, nos scripts SQL, nos testes, nas dependências Python, na configuração dos containers e no dashboard sejam registradas e validadas antes de sua integração ao projeto.

Também busca assegurar a rastreabilidade entre alterações de código, versões do sistema, arquivos processados, estruturas de dados e execuções do pipeline, permitindo identificar quais componentes participaram de determinada versão ou processamento.

1.2 Escopo

Este Plano de Gerenciamento de Configuração aplica-se ao desenvolvimento e à operação acadêmica do FundScope, projeto individual responsável por coletar, tratar, armazenar e disponibilizar informações públicas sobre fundos de investimento.

O escopo do MVP compreende:

- Coleta dos dados cadastrais de fundos publicados pela CVM;
- Processamento dos dados referentes aos últimos 12 meses;
- Download e descompactação dos arquivos disponibilizados em formato ZIP;
- Armazenamento dos arquivos originais na camada Bronze;
- Leitura e tratamento dos arquivos CSV com Python e Pandas;
- Validação da estrutura e dos campos obrigatórios;
- Conversão de datas e valores;
- Tratamento de registros duplicados;
- Relacionamento entre o cadastro e os informes diários;
- Armazenamento dos dados tratados na camada Silver;
- Criação de indicadores e tabelas analíticas na camada Gold;
- Persistência das informações no PostgreSQL;
- Orquestração das tarefas pelo Apache Airflow;

- Registro das execuções e dos arquivos processados;
- Implementação de testes automatizados;
- Gerenciamento e versionamento de dependências Python e do ambiente de desenvolvimento;
- Criação de ambiente reproduzível com Docker;
- Automação do processo de integração com GitHub Actions;
- Desenvolvimento de um dashboard em Streamlit;
- Documentação da instalação, execução e recuperação do sistema.

Não fazem parte do MVP:

- Processamento de todo o histórico disponível na CVM;
- Utilização de todos os conjuntos de dados do Portal da CVM;
- Análise da carteira individual de cada fundo;
- Recomendação de investimentos;
- Previsão de rentabilidade;
- Comparação automática entre fundos para recomendar aplicações;
- Desenvolvimento de aplicativo móvel;
- Cadastro e autenticação de usuários;
- Implantação em infraestrutura de alta disponibilidade;
- Utilização de Kubernetes;
- Utilização de Terraform;
- Atualização dos dados em tempo real.

As funcionalidades não contempladas no MVP poderão ser avaliadas como evoluções futuras após a validação da primeira versão do produto.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
API	Interface utilizada para comunicação e troca de dados entre sistemas.
CVM	Comissão de Valores Mobiliários
CI	Continuous Integration - Integração Contínua
CD	Continuous Deploy/Delivery -Entrega ou implantação contínua
DAG	Representação das tarefas e dependências de um fluxo no Apache Airflow
ETL	Extração, transformação e carga de dados.
MVP	Produto mínimo viável correspondente à primeira versão funcional
PR	Pull Request - Solicitação para integrar alterações de uma branch ao código principal.
ADR	Architecture Decision Record — registro de uma decisão relevante de arquitetura.
Pipeline	Sequência automatizada de tarefas para processamento de dados.
Baseline	Conjunto identificado e estável de itens que formam uma versão.
Idempotência	Capacidade de reprocessar uma carga sem duplicar dados.
Quality gate	Critério obrigatório para aprovação de uma mudança.
Rollback	Retorno para uma versão anterior do sistema.
Schema	Estrutura dos campos e tipos de dados de um arquivo ou banco.

Camada Bronze	Armazenamento dos dados originais, sem transformação.
Camada Silver	Armazenamento dos dados limpos, tipados e padronizados.
Camada Gold	Dados organizados e preparados para análise e consumo.
Captação líquida	Diferença entre o valor das captações e dos resgates.
Patrimônio líquido	Valor do patrimônio informado pelo fundo em determinada data

1.4 Referências

- Comissão de Valores Mobiliários. Portal de Dados Abertos.
  Portal de Dados Abertos da CVM
- Comissão de Valores Mobiliários. Informes Diários de Fundos de Investimento.
  Conjunto de informes diários
- Apache Software Foundation. Documentação oficial do Apache Airflow.
  Apache Airflow
- PostgreSQL Global Development Group. Documentação oficial do PostgreSQL.
  PostgreSQL
- Pandas Development Team. Documentação oficial do Pandas.
  Pandas
- Streamlit. Documentação oficial.
  Streamlit
- Docker. Documentação oficial do Docker Compose.
  Docker Compose
- GitHub. Documentação oficial do GitHub Actions.
  GitHub Actions
- Semantic Versioning. Especificação do Versionamento Semântico.
  Semantic Versioning
- Conventional Commits. Especificação para mensagens de commit.
  Conventional Commits
- Material didático da disciplina de Gerenciamento, Configuração e Processos de Software.
- Template do Plano de Gerenciamento de Configuração baseado no RUP e adaptado para práticas DevOps.

1.5 Visão Geral

O FundScope será estruturado em três fluxos complementares. O pipeline de dados será responsável por coletar, validar, transformar e armazenar os dados da CVM. O dashboard será responsável por consultar e apresentar as informações da camada Gold. O pipeline de CI/CD será responsável por validar as alterações no software, executar os testes e disponibilizar novas versões do projeto.

O Apache Airflow será responsável pela execução e pelo monitoramento do pipeline de dados. O PostgreSQL armazenará os dados tratados e analíticos. O Streamlit disponibilizará a camada de apresentação. O GitHub Actions executará os controles de integração e entrega contínuas. Essa separação permitirá desenvolver, testar e operar cada componente de maneira independente e rastreável.

2. Cultura DevOps e Papéis na Gerência de Configuração

No projeto FundScope, por se tratar de um projeto acadêmico individual, as funções relacionadas à Gerência de Configuração, desenvolvimento, engenharia de dados, DevOps, testes e operação serão exercidas pelo próprio autor do projeto.

Papel	Responsabilidades	Responsável
Gerente de Projeto / Product Owner	Prioriza o backlog, autoriza mudanças de maior impacto e acompanha indicadores de entrega.	Arthur Faust
Engenheiro DevOps / Release Engineer	Projeta e mantém os pipelines de CI/CD, define a estratégia de *branching* e versionamento, administra o registro de artefatos/imagens.	Arthur Faust
Engenheiro de Dados	Implementa as etapas de extração, transformação, validação e carga dos dados e dashboard.	Arthur Faust
QA / Automação de Testes	Mantém a suíte de testes automatizados (unitários, integração, aceitação) executada nos pipelines de CI e CD.	Arthur Faust
SRE / Operações	Monitora ambientes de produção, acompanha as execuções do Airflow e tratamento de falhas.	Arthur Faust

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Categoria	Ferramenta	Finalidade
Controle de versão	Git + GitHub	Versionamento do código, DAGs, testes, configurações e documentação.
CI/CD	GitHub Actions	Automação de build, testes, análise e implantação.
Engenharia de Dados	Python / Pandas	Extração, tratamento e transformação dos dados.
Orquestração de Dados	Apache Airflow	Agendamento, dependências e monitoramento das etapas do pipeline.
Banco de dados	PostgreSQL	Persistência de camadas tratadas e analíticas
Containerização	Docker + Docker Compose	Criação de ambiente reproduzível para Airflow, PostgreSQL e aplicação.
Testes automatizados	Pytest	Execução dos testes unitários e de integração do pipeline de dados.
Qualidade de código	Ruff	Verificação automática de padrões de código Python e identificação de problemas simples antes do merge.
Dashboard	Streamlit	Apresentação dos indicadores e informações analíticas.
Rastreamento de mudanças	GitHub Issues	Registro de funcionalidades, defeitos e alterações relevantes.
Ambiente e dependências Python	uv	Criação do ambiente virtual, instalação das dependências e geração do lockfile.
Dependências / Vulnerabilidades	Dependabot	Identificação e proposta de atualização das dependências.

3.2 Ambientes e Infraestrutura

Os ambientes serão definidos, sempre que possível, por configurações versionadas, garantindo paridade entre eles e permitindo sua recriação de forma rápida e rastreável.

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento	Implementação e testes locais do código, transformações de dados, DAGs, consultas SQL e dashboard. Os serviços necessários serão executados principalmente por Docker Compose.	Criação ou atualização de branch de desenvolvimento.
Homologação / Staging	Validação integrada do pipeline, incluindo execução das DAGs, conexão com PostgreSQL, testes das transformações e verificação do dashboard.	Pull Request aprovado pelo pipeline de CI.
Produção	Representa a versão estável do FundScope, preparada para demonstração, avaliação e eventual disponibilização.	Merge na main, pipeline aprovado e criação de uma release estável.

3.3 Estratégia de Branching e Versionamento

O FundScope adotará uma estratégia de Trunk-Based Development, com branches de curta duração, de forma a favorecer integração frequente das alterações e reduzir conflitos entre versões do código.

A branch principal será a main, que deverá representar sempre a versão integrada e estável do projeto. Alterações serão desenvolvidas em branches temporárias e posteriormente integradas à branch principal por meio de Pull Request.

Serão utilizadas as seguintes convenções:

- **main** — branch principal do projeto, mantida em estado estável e com integração condicionada à aprovação do pipeline de CI;
- **feature/<descricao-curta>** — utilizada para o desenvolvimento de novas funcionalidades, como novas etapas do pipeline, transformações ou recursos do dashboard;
- **release/x.y** — utilizada opcionalmente quando houver necessidade de estabilizar uma versão antes de sua disponibilização;
- **hotfix/<descricao>** — utilizada para correções urgentes em uma versão já disponibilizada, com posterior integração da correção à branch principal.

As branches de desenvolvimento deverão possuir curta duração, sendo integradas à main assim que a funcionalidade estiver concluída e validada.

O versionamento das releases seguirá o padrão Semantic Versioning (SemVer), no formato MAJOR.MINOR.PATCH. As versões serão identificadas por tags Git, por exemplo v1.0.0, v1.1.0 e v1.1.1.

Exemplos:

feature/carga-informes-cvm
feature/camada-silver
release/1.0
hotfix/correcao-carga-duplicada

As mensagens de commit seguirão o padrão Conventional Commits, utilizando tipos como feat, fix, chore, docs, test, refactor e ci, permitindo melhor identificação das mudanças e futura geração automatizada de changelog e notas de versão.

Exemplos:

feat(extract): adiciona coleta dos informes da CVM
feat(silver): implementa tratamento dos dados
fix(load): corrige duplicidade na carga do PostgreSQL
test(transform): adiciona testes das transformacoes
docs(readme): documenta execucao do projeto
ci(github): configura pipeline de integracao continua

4. Identificação da Configuração

4.1 Itens de Configuração

Além do código-fonte, serão considerados itens formais de configuração do FundScope os artefatos necessários para desenvolver, testar, executar e reproduzir o projeto. Esses itens serão versionados no repositório Git sempre que aplicável.

Serão considerados itens de configuração:

- Código-fonte Python responsável pelas etapas de extração, validação, transformação e carga dos dados;
- Código-fonte do dashboard desenvolvido em Streamlit;
- DAGs e arquivos de configuração do Apache Airflow;
- Código e arquivos dos testes automatizados;
- Definições do pipeline de CI/CD armazenadas em .github/workflows;
- Dockerfile e arquivos do Docker Compose utilizados para criação do ambiente reproduzível;
- Scripts SQL utilizados para criação e evolução das estruturas do PostgreSQL;
- Definições de schema e regras de validação dos dados;
- Arquivo pyproject.toml, contendo as dependências e configurações do projeto Python;
- Arquivo uv.lock, contendo as versões resolvidas das dependências utilizadas pelo projeto;
- Arquivos de configuração por ambiente que não contenham informações sensíveis;
- Arquivo .env.example, utilizado para documentar as variáveis de ambiente necessárias;
- Documentação técnica, incluindo README, documentação de instalação e execução e registros de decisões arquiteturais (ADRs), quando aplicáveis.

Senhas, tokens, chaves de acesso e demais informações sensíveis não serão armazenadas no repositório. Esses valores serão mantidos por meio de variáveis de ambiente e mecanismos apropriados de gerenciamento de segredos.

Os arquivos de dados obtidos da CVM não serão versionados diretamente pelo Git. Os arquivos originais serão preservados na camada Bronze, enquanto o código, os schemas, as regras de transformação e os metadados necessários para identificar os dados processados permanecerão sob controle de configuração.

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature	feature/<descrição-curta>	feature/carga-informes-cvm
Branch de release	release/<major>.<minor>	release/1.0
Branch de hotfix	hotfix/<descricao-curta>	hotfix/correcao-carga-duplicada

Item	Convenção	Exemplo
Commit	Conventional Commits: <tipo>(<escopo>): <descricao>	feat(silver): adiciona tratamento dos informes
Tag de release	Versionamento Semântico vX.Y.Z	v1.0.0
DAG do Airflow	fundscope_<processo>	fundscope_informes_diarios
Imagem do container	fundscope/<servico>:<versao>	fundscope/app:1.0.0
ADR	NNNN-<descricao-curta>.md	0001-arquitetura-medallion.md

4.3 Estrutura do Repositório

```
fundscope/
├── .github/
│   └── workflows/
│       ├── ci.yml
│       └── cd.yml
│
├── src/
│   └── fundscope/
│       ├── extract/
│       │   └── cvm.py
│       ├── transform/
│       │   └── informes.py
│       ├── load/
│       │   └── postgres.py
│       ├── validation/
│       │   ├── schema.py
│       │   └── informes.py
│       ├── pipeline.py
│       └── healthcheck.py
│
├── sql/
│   └── schema.sql
│
├── tests/
│   └── unit/
│       ├── test_schema.py
│       └── test_informes.py
│
├── Dockerfile
├── docker-compose.yml
├── pyproject.toml
├── uv.lock
├── .python-version
├── .env.example
├── .env.staging
├── .env.production
├── .gitignore
└── README.md
```

5. Integração Contínua (CI)

Todo Pull Request para a branch main deve disparar automaticamente um pipeline de CI. Nenhuma mudança é integrada sem que o pipeline seja concluído com sucesso.

5.1 Etapas do Pipeline de CI

1. Checkout do código fonte
2. Preparação do ambiente Python e instalação das dependências
3. Análise estática e verificação da qualidade do código
4. Testes unitários, com medição de cobertura mínima.
5. Testes de integração.
6. Verificação de dependências e segurança
7. Validação da construção dos containers

5.2 Política de Pull Request e Revisão de Código

- Todo Pull Request destinado à branch main deverá ser revisado antes do merge.
- Por se tratar de um projeto acadêmico individual, não será exigida aprovação de outro desenvolvedor; a revisão será realizada pelo próprio responsável pelo projeto.
- Os Pull Requests deverão ser pequenos e de curta duração, para reduzir conflitos e facilitar a análise das alterações.
- O Pull Request deverá conter uma descrição clara da mudança realizada e, quando aplicável, estar relacionado a uma Issue do projeto.
- O merge somente será permitido quando o pipeline de CI estiver concluído com sucesso, atendendo aos critérios de qualidade definidos para o projeto.
- Alterações que resultem em falha nos testes, erros de lint ou outras verificações obrigatórias não deverão ser integradas à branch main.

5.3 Critérios de Qualidade (Quality Gate)

- Cobertura mínima de testes automatizados de 80% nos módulos críticos, especialmente nas rotinas de transformação e validação de dados.
- Todos os testes unitários e de integração devem ser concluídos com sucesso.
- Zero erros bloqueantes identificados pelo Ruff.
- Zero vulnerabilidades críticas ou altas identificadas nas dependências do projeto.
- O processo de build dos containers Docker deve ser concluído com sucesso.
- As validações de dados obrigatórias devem ser aprovadas, incluindo verificação de colunas obrigatórias, tipos de dados esperados e tratamento de registros duplicados.
- O pipeline de CI do GitHub Actions deve ser concluído integralmente com sucesso antes do merge na branch main.

- Caso qualquer critério obrigatório falhe, o Pull Request não deverá ser integrado até que o problema seja corrigido.

6. Entrega e Implantação Contínua (CD)

Após a publicação do artefato pelo pipeline de CI, o pipeline de CD é responsável por promover essa mesma versão, sem recompilação, através dos ambientes até a produção.

6.1 Etapas do Pipeline de CD

1. Deploy automático em ambiente de Homologação/Staging.
2. Execução de testes automatizados de aceitação/E2E em staging.
3. Gate de aprovação para produção (automático ou aprovação manual, conforme criticidade).
4. Deploy da versão estável em produção.
5. Testes de fumaça (smoke tests) pós-deploy e verificação de saúde (health checks).
6. Geração automática de baseline/tag de release e notas de versão (changelog).

6.2 Estratégias de Implantação

Estratégia	Descrição
Substituição controlada da versão	Estratégia adotada no MVP. A versão anterior será substituída pela nova versão somente após a aprovação dos testes em Homologação/Staging.
Rolling update	Não será utilizado inicialmente, pois o MVP não prevê múltiplas instâncias da aplicação.
Blue-Green	Não será utilizado no MVP. Poderá ser avaliado futuramente caso o sistema seja disponibilizado em infraestrutura de produção com maior disponibilidade.
Canary Release	Não será utilizado no MVP, pois não haverá distribuição gradual de tráfego entre diferentes versões.
Feature Flags	Poderão ser avaliadas futuramente caso haja necessidade de ativar funcionalidades de forma independente do deploy.

Para o MVP do FundScope será adotada uma estratégia simples de substituição controlada da versão. Uma nova versão somente será disponibilizada após sua validação no ambiente de Homologação/Staging. A versão anterior permanecerá identificada no repositório por meio de tag Git, permitindo seu restabelecimento em caso de falha.

Estratégias avançadas de implantação, como Blue-Green e Canary Release, não serão implementadas inicialmente devido ao caráter acadêmico e à arquitetura simplificada do projeto.

6.3 Rollback e Recuperação

Toda implantação do FundScope deverá permitir retorno à última versão considerada estável.

Em caso de falha após a implantação, o rollback será realizado utilizando a tag Git correspondente à versão anterior, permitindo restaurar o código, as configurações e as dependências associadas àquela baseline.

Os containers poderão ser reconstruídos a partir da versão estável anterior e novamente executados por meio do Docker Compose.

Em relação aos dados, os arquivos originais armazenados na camada Bronze serão preservados sem transformação, permitindo o reprocessamento das camadas Silver e Gold caso uma falha seja identificada durante o processamento. Os arquivos originais da camada bronze serão preservados permitindo o reprocessamento das camadas Silver e Gold em caso de falha.

Alterações na estrutura do PostgreSQL deverão possuir procedimento documentado de recuperação e, quando necessário, backup antes da aplicação de mudanças que possam comprometer os dados existentes.

O rollback poderá ser acionado quando forem identificadas falhas nos testes de fumaça, indisponibilidade dos serviços, erros nas DAGs, problemas de conexão com o banco de dados ou falhas que comprometam a integridade do processamento.

7. Infraestrutura como Código (IaC)

No MVP do FundScope não será utilizada uma ferramenta dedicada de Infraestrutura como Código, como Terraform. A reprodução do ambiente será realizada por meio de Dockerfile, docker-compose.yml e arquivos de configuração versionados no repositório Git.

Esses arquivos seguirão o mesmo fluxo de controle de mudanças aplicado ao código-fonte. Informações sensíveis, como senhas e tokens, não serão versionadas e serão fornecidas por variáveis de ambiente.

Caso o projeto evolua para uma infraestrutura em nuvem, poderá ser adotada futuramente uma ferramenta específica de IaC.

7.1 Versionamento de Infraestrutura

- Alterações no Dockerfile, docker-compose.yml e demais arquivos de configuração do ambiente deverão seguir o mesmo processo de controle de versão utilizado para o código-fonte.
- As alterações deverão ser realizadas em branch apropriada e integradas à main por meio de Pull Request.
- O pipeline de CI deverá validar, sempre que aplicável, a construção dos containers antes da integração da alteração.
- As configurações necessárias à reprodução do ambiente deverão permanecer versionadas no repositório Git.
- Arquivos contendo credenciais, senhas, tokens ou outros dados sensíveis não deverão ser versionados.
- O arquivo .env.example poderá ser utilizado para documentar as variáveis de ambiente necessárias, sem armazenar valores reais.
- As versões estáveis das configurações de infraestrutura estarão associadas às mesmas tags e baselines utilizadas para as releases do FundScope.

7.2 Ambientes Efêmeros

A criação automática de ambientes efêmeros para cada Pull Request não será implementada no MVP do FundScope, considerando o caráter acadêmico e a infraestrutura simplificada adotada pelo projeto.

A validação das alterações será realizada por meio do pipeline de CI e dos ambientes definidos para Desenvolvimento e Homologação/Staging.

O uso de ambientes efêmeros poderá ser avaliado futuramente caso o FundScope seja implantado em infraestrutura de nuvem e passe a necessitar de ambientes isolados para validação de mudanças.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

As mudanças no FundScope serão registradas e rastreadas por meio do GitHub. Alterações de rotina, como novas funcionalidades, correções e ajustes técnicos, seguirão o fluxo:

GitHub Issue → branch → desenvolvimento → Pull Request → pipeline de CI → merge na main → pipeline de CD.

O histórico de Issues, commits, Pull Requests e execuções dos pipelines constituirá o registro formal das mudanças realizadas no projeto.

8.2 Comitê de Controle de Mudança (CCM)

Por se tratar de um projeto acadêmico individual, o FundScope não possuirá um Comitê de Controle de Mudanças formal.

Mudanças de maior impacto deverão receber análise específica antes da implementação, especialmente:

- Alterações estruturais no banco de dados;
- Mudanças relevantes na arquitetura do pipeline;
- Alterações nas configurações de infraestrutura;
- Mudanças que afetem a estrutura ou o processamento dos dados;
- Hotfixes ou exceções ao fluxo padrão de desenvolvimento.

Essas mudanças deverão ser registradas em GitHub Issue e documentadas no respectivo Pull Request. Decisões arquiteturais relevantes poderão também ser registradas por meio de ADRs.

9. Baselines e Releases

9.1 Geração Automática de Baseline

Cada versão estável do FundScope será registrada como uma baseline por meio de uma tag Git seguindo o padrão de Versionamento Semântico (SemVer).

A baseline deverá permitir identificar o commit correspondente, os arquivos de configuração, as dependências registradas no uv.lock e as definições de ambiente utilizadas naquela versão.

A primeira versão completa do MVP será identificada como v1.0.0.

9.2 Notas de Versão (Release Notes)

As notas de versão serão publicadas junto à release no GitHub e deverão apresentar as principais funcionalidades, correções e alterações realizadas desde a versão anterior.

Sempre que possível, essas informações serão geradas automaticamente a partir das mensagens de commit padronizadas com Conventional Commits.

Cada release ficará associada à respectiva tag de versão, permitindo rastrear as alterações que compõem aquela baseline.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

O acompanhamento do FundScope será realizado por meio dos recursos de monitoramento e logs das ferramentas utilizadas no projeto.

O Apache Airflow permitirá acompanhar a execução das DAGs, o status das tarefas, o tempo de execução e eventuais falhas no pipeline de dados. O GitHub Actions permitirá acompanhar o histórico de execução dos pipelines de CI/CD.

As cargas de dados também deverão registrar informações como arquivo processado, período de referência, data da execução, quantidade de registros processados e status da carga.

10.2 Métricas DORA

Métrica	Descrição
Deployment Frequency	Frequência com que o código é implantado em produção.
Lead Time for Changes	Tempo entre o commit e a implantação em produção.
Change Failure Rate	Percentual de implantações que resultam em falha em produção.
Mean Time to Restore (MTTR)	Tempo médio para restaurar o serviço após uma falha.

As métricas DORA serão utilizadas de forma simplificada para fins acadêmicos, considerando que o FundScope é um projeto individual e não possui inicialmente operação contínua em ambiente produtivo público.

10.3 Relatórios e Auditorias

O histórico do Git, das Issues, dos Pull Requests, dos pipelines do GitHub Actions, das releases e das execuções do Apache Airflow constituirá o principal conjunto de evidências para auditoria do gerenciamento de configuração.

Esses registros permitirão identificar as alterações realizadas, o responsável, a versão correspondente, o resultado dos testes e o histórico das execuções.

Dessa forma, não será necessária a geração manual de relatórios periódicos de configuração, salvo quando exigido para fins acadêmicos ou de documentação do projeto.

11. Marcos

Marco	Evidência Esperada
Criação do repositório do projeto	Repositório GitHub criado e estruturado

Estrutura inicial do FundScope	Diretórios, arquivos básicos e README disponíveis
Configuração do ambiente Python	pyproject.toml e uv.lock versionados
Ambiente reproduzível com Docker	Dockerfile e docker-compose.yml funcionais
Implantação do pipeline de CI	GitHub Actions executando lint e testes automaticamente
Primeira execução integrada do pipeline de dados	DAG do Airflow executada com sucesso
Integração com PostgreSQL	Dados tratados persistidos no banco
Implementação das camadas Bronze, Silver e Gold	Fluxo de processamento funcionando entre as três camadas
Disponibilização do dashboard	Aplicação Streamlit consumindo dados analíticos
Implantação do pipeline de CD	Fluxo de promoção para homologação e release configurado
Primeira release estável	Tag v1.0.0 criada no repositório
Entrega do MVP	Pipeline completo, documentado e reproduzível

12. Treinamento e Recursos

Como o FundScope é um projeto acadêmico individual, as atividades de capacitação serão realizadas pelo próprio responsável ao longo do desenvolvimento.

Serão necessários conhecimentos nas principais ferramentas e tecnologias adotadas pelo projeto, incluindo:

- Python e Pandas para processamento e transformação de dados;
- SQL e PostgreSQL para persistência e consulta;
- Apache Airflow para orquestração e monitoramento do pipeline;
- Git e GitHub para controle de versão;
- GitHub Actions para automação dos pipelines de CI/CD;
- Docker e Docker Compose para criação do ambiente reproduzível;
- uv para gerenciamento do ambiente e das dependências Python;
- Pytest para testes automatizados;
- Ruff para análise de qualidade do código;
- Streamlit para desenvolvimento do dashboard.

Serão utilizadas prioritariamente as documentações oficiais das ferramentas, os materiais disponibilizados na disciplina e exemplos práticos desenvolvidos durante a implementação do projeto.

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

O FundScope não prevê a utilização de software desenvolvido por subcontratados. As dependências externas do projeto serão compostas principalmente por bibliotecas, ferramentas de código aberto e serviços utilizados no ambiente de desenvolvimento e execução.

As dependências Python serão declaradas no arquivo pyproject.toml e suas versões resolvidas serão registradas no arquivo uv.lock, permitindo a reprodução do ambiente utilizado pelo projeto.

- As dependências serão versionadas e mantidas sob controle de configuração;
- Alterações de dependências deverão seguir o mesmo fluxo de branch, Pull Request e pipeline de CI utilizado para mudanças no código;
- O Dependabot será utilizado para identificar atualizações e vulnerabilidades conhecidas nas dependências;
- Atualizações propostas deverão ser validadas pelos testes automatizados antes do merge;
- Dependências que apresentem vulnerabilidades críticas ou altas deverão ser corrigidas ou atualizadas antes da integração à branch main.

A Comissão de Valores Mobiliários (CVM) será considerada uma fonte externa de dados do projeto. Como a estrutura dos arquivos disponibilizados poderá sofrer alterações, o pipeline deverá validar o schema, os campos obrigatórios e os formatos esperados antes do processamento, evitando que mudanças na fonte comprometam as etapas posteriores do pipeline.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção:

Etapa	Descrição	Gatilho
CI	Preparação do ambiente com uv, execução do Ruff, testes unitários e de integração com Pytest, verificação das dependências e validação do build dos containers Docker.	Abertura/atualização de Pull Request
Merge	Integração da alteração à branch main após aprovação dos critérios de qualidade.	Pipeline de CI aprovado
CD — Staging	Disponibilização da versão em Homologação/Staging, com testes de integração, aceitação e verificação dos principais componentes do FundScope.	Merge na branch main
Gate de Release	Validação da versão em homologação antes da promoção para produção/release estável.	Sucesso dos testes em staging
CD — Produção	Disponibilização da versão estável, execução de smoke tests e geração da baseline, tag SemVer e notas de versão.	Aprovação do gate de release